AWS IAM & CLOUD SECURITY

PROJECT DOCUMENTATION

Identity & Access Management • S3 Bucket Policies • EC2 Instances • CloudTrail • KMS Encryption • DynamoDB •

Groups: Facebook | Instagram | Threads | IT Users: 8 Total (2 per group) Services: 8 AWS Services

4 IAM Groups	**8** IAM Users (2 per group)	**8** AWS Services Used	**3** S3 Buckets + EC2 Instances

Date: 19 July 2026

Table of Contents

1. Executive Summary3
2. Project Overview & AWS Services Used4
3. Phase 1 Root Account Security: Enabling MFA5
4. Phase 2 Admin IAM User Creation6
5. Phase 3 IAM Groups & Users7
   5.1 Creating the Four Groups7
   5.2 Creating & Assigning Users8
6. Phase 4 S3 Buckets & Inline Access Policies9
   6.1 Bucket Creation9
   6.2 Group-Specific Inline Policies (S3)10
   6.3 IT Group Full S3 Access Policy12
7. Phase 5 EC2 Instances & Access Control13
8. Phase 6 CloudTrail, SNS & EventBridge Alerting15
   8.1 CloudTrail Management Events Trail15
   8.2 SNS Topic & Email Subscription16
   8.3 EventBridge Rule S3 Bucket Create/Delete Alert17
9. Phase 7 KMS Encryption & DynamoDB19
   9.1 KMS Symmetric Key Creation19
   9.2 DynamoDB Table with KMS Encryption20
   9.3 User Access Control on DynamoDB21
10. IAM Least Privilege Summary24
11. Key Findings & AWS Security Best Practices25
12. Screenshot Evidence Index26

1. Executive Summary

This document presents the complete technical documentation of an end-to-end AWS Identity and Access Management (IAM) and cloud security project. The project demonstrates practical, hands-on proficiency across eight core AWS services IAM, S3, EC2, CloudTrail, SNS, EventBridge, KMS, DynamoDB, and Amazon Bedrock applied in a structured, security-first workflow that mirrors real-world cloud administration practices.

The project simulated a media organisation with four business groups Facebook, Instagram, Threads, and IT each with two IAM users. Beginning with root account hardening (MFA enablement), an administrative IAM user was created to avoid operating as root. The admin account was then used to provision all downstream resources: three S3 buckets and EC2 instances (one per non-IT group), with group-specific inline policies restricting each group to only their assigned resources. The IT group was granted full access to all resources across the environment.

An automated alerting pipeline was built using CloudTrail (for management event logging), SNS (for email notification), and EventBridge (to trigger an alert whenever an S3 bucket is created or deleted). Data at rest was protected using a KMS symmetric customer managed key linked to a DynamoDB table, with user-level access controls defined on the table.

2. Project Overview & AWS Services Used

2.1 Project Context

This project is structured as a sequential, eight-phase build, with each phase introducing an additional AWS service and a corresponding security control. The order of operations mirrors best-practice cloud deployment: secure the root account first, establish administrative access, provision resources, apply access policies, implement monitoring and alerting, encrypt data, and finally deploy application-layer services.

2.2 AWS Services Used

AWS IAM	Identity & Access Management groups, users, policies, MFA. The foundation of all access control in this project.
Amazon S3	Simple Storage Service three object storage buckets created for Facebook, Instagram, and Threads groups.
Amazon EC2	Elastic Compute Cloud virtual machine instances created per group for compute resource access control testing.
AWS CloudTrail	Management event trail capturing all API calls made in the AWS account the audit log backbone.
Amazon SNS	Simple Notification Service topic and email subscription created to deliver S3 event alerts.
Amazon EventBridge	Event routing service rule created to link CloudTrail S3 API events to the SNS topic for email alerts.
AWS KMS	Key Management Service symmetric customer managed key (CMK) created for DynamoDB table encryption.
Amazon DynamoDB	Managed NoSQL database table created and encrypted with KMS CMK; user-level access policies defined.

2.3 Organisational Structure

Group	Users	S3 Bucket	EC2 Instance	Access Level
Facebook	fb-user-1, fb-user-2	`s3-facebook-bucket`	`ec2-facebook`	Own bucket + EC2 only
Instagram	ig-user-1, ig-user-2	`s3-instagram-bucket`	`ec2-instagram`	Own bucket + EC2 only

Threads	th-user-1, th-user-2	`s3-threads-bucket`	`ec2-threads`	Own bucket + EC2 only
IT	it-user-1, it-user-2	**All 3 buckets**	**All 3 instances**	**Full access to ALL resources**

3. Phase 1 Root Account Security: Enabling MFA

AWS IAM	Service: IAM > Security Credentials > Assign MFA device

The first action taken upon accessing the newly created AWS account was to secure the root user account with Multi-Factor Authentication (MFA). The root user in AWS has unrestricted, irrevocable access to every resource and service in the account it cannot be restricted by IAM policies. Compromising root credentials is a catastrophic event, and MFA provides a critical second factor of authentication that makes credential theft alone insufficient for an attacker to gain access.

Why Root Account Security Is the Most Critical First Step

The AWS root user has full control over the entire account billing, IAM, all services. AWS documentation and the Well-Architected Framework explicitly state:

- Never use the root account for day-to-day operations
- Enable MFA on the root account immediately after account creation
- Store root credentials offline in a secure location and use them only for tasks that absolutely require root (e.g. changing account settings, enabling IAM access to billing)
- Create an IAM admin user for all administrative tasks this project follows this pattern exactly in Phase 2

3.1 MFA Enablement Steps

1. Logged into the AWS Management Console as the root user
2. Clicked the account name (top right) > Security credentials
3. Scrolled to the Multi-factor authentication (MFA) section and clicked Assign MFA device
4. Selected MFA device type: Authenticator app (virtual MFA device)
5. Opened the authenticator app on a mobile device (Google Authenticator / Authy)
6. Scanned the QR code displayed by AWS using the authenticator app
7. Entered two consecutive 6-digit TOTP codes from the authenticator app to complete verification
8. Clicked Add MFA MFA was successfully enabled on the root account

📷 **[SCREENSHOT]**

AWS Console Root Account MFA Assigned (Security Credentials Page)

Capture the AWS IAM Security Credentials page showing the MFA section with the status 'Active' and the MFA device listed (e.g. authenticator app). Show the green checkmark or 'Assigned' status confirming MFA is enabled.

4. Phase 2 Admin IAM User Creation

AWS IAM	Service: IAM > Users > Create user > Attach AdministratorAccess policy

With the root account secured, an administrative IAM user was created to handle all subsequent operations. This follows the AWS security best practice of never performing day-to-day tasks as root. The IAM admin user was granted the AWS managed policy AdministratorAccess which provides full access to all AWS services and resources but unlike root, this access can be revoked, restricted, or monitored via CloudTrail.

4.1 Creating the Admin IAM User

9. Logged in as root, navigated to IAM > Users > Create user
10. Set username: admin-user (or [insert actual username used])
11. Selected: Provide user access to the AWS Management Console
12. Selected: I want to create an IAM user (not Identity Center)
13. Set console password: Custom password (strong, stored securely)
14. Unchecked 'Users must create a new password at next sign-in' for immediate admin use
15. On the permissions page, selected Attach policies directly
16. Searched for and attached: AdministratorAccess (AWS managed policy)
17. Reviewed and created the user
18. Downloaded or noted the sign-in URL and credentials
19. Signed out of root and signed back in as admin-user for all remaining project phases

Admin User Attribute	Value
Username	admin-user
Access Type	AWS Management Console access
Policy Attached	AdministratorAccess (AWS Managed Policy)
Policy ARN	arn:aws:iam::aws:policy/AdministratorAccess
MFA on Admin User	Recommended enable MFA on admin-user as additional hardening
Used For	All subsequent project phases root not used again

📷 **[SCREENSHOT]**

AWS IAM Admin User Created with AdministratorAccess Policy

Capture the IAM User detail page for admin-user, showing the user summary (username, ARN, creation date) and the Permissions tab confirming AdministratorAccess policy is attached.

5. Phase 3 IAM Groups & Users

AWS IAM	Service: IAM > User groups > Create group | IAM > Users > Create user > Add to group

5.1 Creating the Four Groups

Four IAM user groups were created to represent the four business units in the organisation. IAM groups are the recommended mechanism for managing permissions at scale policies are attached to the group, and users inherit permissions through their group membership. This avoids the anti-pattern of attaching policies directly to individual users.

20. Navigated to IAM > User groups > Create group
21. Created group: Facebook no permissions attached at this stage (inline policies added in Phase 4)
22. Created group: Instagram same approach
23. Created group: Threads same approach
24. Created group: IT same approach (permissions added in Phase 4 & 5)

Group Name	Group ARN (pattern)	Members	Permissions Source
Facebook	`arn:aws:iam::[account]:group/Facebook`	fb-user-1, fb-user-2	Inline policy S3 + EC2 (own resources only)
Instagram	`arn:aws:iam::[account]:group/Instagram`	ig-user-1, ig-user-2	Inline policy S3 + EC2 (own resources only)
Threads	`arn:aws:iam::[account]:group/Threads`	th-user-1, th-user-2	Inline policy S3 + EC2 (own resources only)
IT	`arn:aws:iam::[account]:group/IT`	it-user-1, it-user-2	Inline policy full S3 + EC2 access (all resources)

📷 **[SCREENSHOT]**

AWS IAM All Four Groups Created (User Groups List)

Capture the IAM > User groups page showing all four groups listed: Facebook, Instagram, Threads, IT with their member counts visible in the table.

5.2 Creating & Assigning Users

Two IAM users were created for each group (8 users total). Each user was created with console access and programmatic access credentials as appropriate, then added to their corresponding group during creation.

25. Navigated to IAM > Users > Create user for each of the 8 users
26. Set usernames following the pattern: fb-user-1, fb-user-2, ig-user-1, ig-user-2, th-user-1, th-user-2, it-user-1, it-user-2
27. For each user, selected 'Add user to group' and selected the appropriate group
28. Set console passwords and downloaded or noted credentials

Username	Group	Console Access	Permissions (via group)
`fb-user-1`	**Facebook**	Yes	Facebook S3 bucket + EC2 only
`fb-user-2`	**Facebook**	Yes	Facebook S3 bucket + EC2 only
`ig-user-1`	**Instagram**	Yes	Instagram S3 bucket + EC2 only
`ig-user-2`	**Instagram**	Yes	Instagram S3 bucket + EC2 only
`th-user-1`	**Threads**	Yes	Threads S3 bucket + EC2 only
`th-user-2`	**Threads**	Yes	Threads S3 bucket + EC2 only
`it-user-1`	**IT**	Yes	Full access all S3 buckets + all EC2
`it-user-2`	**IT**	Yes	Full access all S3 buckets + all EC2

📷 **[SCREENSHOT]**

AWS IAM Users List Showing All 8 Users

Capture the IAM > Users list showing all 8 users (fb-user-1, fb-user-2, ig-user-1, ig-user-2, th-user-1, th-user-2, it-user-1, it-user-2) and their group column showing group membership.

6. Phase 4 S3 Buckets & Inline Access Policies

Amazon S3 + AWS IAM	Service: S3 > Create bucket | IAM > User groups > [Group] > Permissions > Add inline policy

6.1 Bucket Creation

Three S3 buckets were created from the admin-user account one for each of the three content groups (Facebook, Instagram, Threads). The IT group is not assigned a dedicated bucket; instead, IT has access to all three buckets. Bucket names must be globally unique across all of AWS.

29. Navigated to S3 > Create bucket
30. Created bucket: [your-bucket-name]-facebook (insert your actual bucket name must be globally unique)
31. Region: [Insert region used e.g. us-east-1]
32. All other settings left at default Block Public Access enabled (all four blocks checked)
33. Created bucket: [your-bucket-name]-instagram same settings
34. Created bucket: [your-bucket-name]-threads same settings

Bucket Name	Assigned Group	Region	Access Control
`[prefix]-facebook`	**Facebook**	[insert region]	Facebook group only + IT group (full)
`[prefix]-instagram`	**Instagram**	[insert region]	Instagram group only + IT group (full)
`[prefix]-threads`	**Threads**	[insert region]	Threads group only + IT group (full)

📷 **[SCREENSHOT]**

AWS S3 Three Buckets Created (S3 Buckets List)

Capture the Amazon S3 console showing all three buckets listed: [prefix]-facebook, [prefix]-instagram, [prefix]-threads, with their region and creation date visible.

6.2 Group-Specific Inline Policies (S3)

Inline policies were attached to each group to grant access to only their designated S3 bucket. An inline policy is embedded directly into the group (rather than being a standalone managed policy) this tightly couples the permission to the specific identity and makes it clear that the policy exists only for this group's resource. This enforces least-privilege access: Facebook users can only interact with the Facebook bucket, Instagram users only with the Instagram bucket, and so on.

Facebook Group S3 Inline Policy

Navigated to IAM > User groups > Facebook > Permissions tab > Add permissions > Create inline policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject",
        "s3:DeleteObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::[prefix]-facebook",
        "arn:aws:s3:::[prefix]-facebook/*"
      ]
    }
  ]
}
```

📷 **[SCREENSHOT]**

AWS IAM Facebook Group Inline Policy (S3)

Capture the IAM > User groups > Facebook > Permissions tab showing the inline policy attached, with the policy name visible and the JSON editor showing the Allow S3 actions on the Facebook bucket ARN.

Instagram Group S3 Inline Policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action":
["s3:GetObject","s3:PutObject","s3:DeleteObject","s3:ListBucket"],
      "Resource": [
        "arn:aws:s3:::[prefix]-instagram",
        "arn:aws:s3:::[prefix]-instagram/*"
      ]
    }
  ]
}
```

Threads Group S3 Inline Policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
```

```
      "Effect": "Allow",
      "Action":
["s3:GetObject","s3:PutObject","s3:DeleteObject","s3:ListBucket"],
      "Resource": [
        "arn:aws:s3:::[prefix]-threads",
        "arn:aws:s3:::[prefix]-threads/*"
      ]
    }
  ]
}
```

📷 **[SCREENSHOT]**

AWS IAM Instagram & Threads Group Inline Policies (S3)

Capture the Permissions tabs for the Instagram and Threads groups showing their respective inline policies attached with the correct bucket ARNs in the Resource field.

6.3 IT Group Full S3 Access Policy

The IT group was granted full S3 access across all three buckets. This simulates the role of a cloud administrator or IT operations team who requires visibility and management capability over all departmental storage resources for administrative, backup, and troubleshooting purposes.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "s3:*",
      // Full S3 access   all actions on all buckets
      "Resource": [
        "arn:aws:s3:::[prefix]-facebook",
        "arn:aws:s3:::[prefix]-facebook/*",
        "arn:aws:s3:::[prefix]-instagram",
        "arn:aws:s3:::[prefix]-instagram/*",
        "arn:aws:s3:::[prefix]-threads",
        "arn:aws:s3:::[prefix]-threads/*"
      ]
    }
  ]
}
```

📷 **[SCREENSHOT]**

AWS IAM IT Group Inline Policy: Full S3 Access to All Buckets

Capture the IAM > User groups > IT > Permissions tab showing the IT group's inline policy with s3: action and all three bucket ARNs listed in the Resource array.*

7. Phase 5 EC2 Instances & Access Control

Amazon EC2 + AWS IAM	Service: EC2 > Launch instance | IAM > User groups > [Group] > Add inline policy (EC2)

Three EC2 instances were created from the admin-user account one per content group (Facebook, Instagram, Threads). EC2 instances were configured with minimal resources appropriate for a lab environment (t2.micro free tier eligible). Inline policies were then added to each group granting access only to their specific EC2 instance (identified by instance ID), while the IT group received full EC2 access across all instances.

7.1 EC2 Instance Creation

35. Navigated to EC2 > Instances > Launch instances
36. Set instance name: EC2-Facebook
37. Selected AMI: Amazon Linux 2023 (or Windows Server 2022 insert actual AMI used)
38. Selected Instance type: t2.micro (free tier eligible)
39. Key pair: Created a new key pair or selected existing required for SSH access
40. Network settings: Default VPC, default subnet, allow SSH (port 22) from My IP only
41. Storage: 8 GB gp3 root volume (default)
42. Launched instance noted the instance ID (e.g. i-0abc123456789def0)
43. Repeated for EC2-Instagram and EC2-Threads with identical settings

Instance Name	Assigned Group	Type	AMI	Instance ID
EC2-Facebook	**Facebook**	`t2.micro`	Amazon Linux 2023	`i-[insert ID]`
EC2-Instagram	**Instagram**	`t2.micro`	Amazon Linux 2023	`i-[insert ID]`
EC2-Threads	**Threads**	`t2.micro`	Amazon Linux 2023	`i-[insert ID]`

📷 **[SCREENSHOT]**

AWS EC2 Three Instances Running (Instances List)

Capture the EC2 > Instances page showing all three instances (EC2-Facebook, EC2-Instagram, EC2-Threads) in the 'running' state with their instance IDs, instance types, and availability zones visible.

7.2 Group-Specific EC2 Inline Policies

Each group's inline policy was updated (or a second inline policy was added) to include EC2 access scoped to their specific instance ID. The Resource ARN for EC2 instances follows the format: arn:aws:ec2:[region]:[account-id]:instance/[instance-id].

Facebook Group EC2 Inline Policy (Scoped to EC2-Facebook)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "ec2:StartInstances",
        "ec2:StopInstances",
        "ec2:RebootInstances",
        "ec2:DescribeInstances",
        "ec2:DescribeInstanceStatus"
      ],
      "Resource":
"arn:aws:ec2:[region]:[account-id]:instance/i-[facebook-instance-id]"
    },
    {
      "Effect": "Allow",
      "Action": "ec2:DescribeInstances",
      "Resource": "*"
      // DescribeInstances requires * resource   needed to list instances
    }
  ]
}
```

The same scoped EC2 inline policy pattern was applied to the Instagram and Threads groups, replacing the instance ID with their respective EC2-Instagram and EC2-Threads instance IDs.

IT Group Full EC2 Access Policy (All Instances)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "ec2:*",
      // Full EC2 access   all actions on all instances
      "Resource": "*"
    }
  ]
}
```

📷 **[SCREENSHOT]**

AWS IAM Group Inline Policies: EC2 Access (Facebook, Instagram, Threads, IT)

Capture the Permissions tab of one or more groups (e.g. Facebook group) showing both the S3 and EC2 inline policies listed. Also capture the IT group's Permissions tab showing the full EC2 access policy.

8. Phase 6 CloudTrail, SNS & EventBridge Alerting

8.1 CloudTrail Management Events Trail

AWS CloudTrail	Service: CloudTrail > Create trail > Management events

A CloudTrail trail was created to capture all management events across the AWS account. Management events represent API calls that create, modify, or delete AWS resources exactly the category of activity that should be monitored for security and compliance purposes. The trail logs to an S3 bucket, creating a tamper-evident audit record of every action taken in the account.

44. Navigated to CloudTrail > Trails > Create trail
45. Set trail name: [insert trail name e.g. management-events-trail]
46. Storage location: Created a new S3 bucket for trail logs (or used existing)
47. Log file SSE-KMS encryption: Optional can be enabled with a KMS key
48. CloudWatch Logs: Optional can be linked for real-time log analysis
49. Under Events, selected Management events
50. Read/Write events: All (captures both read and write API calls)
51. Excluded AWS KMS events: Optional (KMS generates high volume exclude if desired)
52. Created the trail CloudTrail began capturing management events immediately

📷 **[SCREENSHOT]**

AWS CloudTrail Trail Created (Management Events)

Capture the CloudTrail > Trails page showing the created trail with its name, S3 bucket destination, and status (Logging: On). Also capture the trail detail showing Management Events selected with Read + Write configured.

8.2 SNS Topic & Email Subscription

Amazon SNS	Service: SNS > Topics > Create topic | SNS > Subscriptions > Create subscription

An SNS (Simple Notification Service) topic was created as the delivery mechanism for S3 event alerts. A subscription was added to the topic linking it to an email address, so that whenever the topic receives a message, it forwards that message as an email to the subscribed address.

Creating the SNS Topic

53. Navigated to SNS > Topics > Create topic
54. Type: Standard (not FIFO order not required for alerts)
55. Name: [insert topic name e.g. s3-bucket-alert-topic]

56. All other settings left at default
57. Created the topic noted the Topic ARN (required for EventBridge rule)

Creating the Email Subscription

58. Navigated to SNS > Subscriptions > Create subscription
59. Topic ARN: Selected the topic created above
60. Protocol: Email
61. Endpoint: [insert email address e.g. admin@organisation.com]
62. Created the subscription a confirmation email was sent to the address
63. Opened the confirmation email and clicked the Confirm subscription link
64. Subscription status changed to Confirmed

📷 **[SCREENSHOT]**

Amazon SNS Topic Created & Email Subscription Confirmed

Capture the SNS > Topics page showing the created topic, and the SNS > Subscriptions page showing the email subscription with Status = 'Confirmed' and the endpoint email address visible.

8.3 EventBridge Rule S3 Bucket Create/Delete Alert

Amazon EventBridge	Service: EventBridge > Rules > Create rule > Event pattern: CloudTrail API Call via S3

An EventBridge rule was created to act as the glue between CloudTrail (the event source) and SNS (the notification target). The rule monitors CloudTrail for specific S3 API calls specifically CreateBucket and DeleteBucket and when either event is detected, it sends a message to the SNS topic, which then delivers the email alert to the subscribed address.

Creating the EventBridge Rule

65. Navigated to EventBridge > Rules > Create rule
66. Name: [insert rule name e.g. s3-bucket-create-delete-alert]
67. Description: Trigger SNS email alert when an S3 bucket is created or deleted
68. Event bus: default
69. Rule type: Rule with an event pattern
70. Event source: AWS events or EventBridge partner events
71. Sample event: AWS CloudTrail
72. Event pattern: Selected 'Custom pattern (JSON editor)' and entered the pattern below

```
{
  "source": ["aws.s3"],
  "detail-type": ["AWS API Call via CloudTrail"],
  "detail": {
    "eventSource": ["s3.amazonaws.com"],
```

```
      "eventName": ["CreateBucket", "DeleteBucket"]
    }
  }
```

73. Clicked Next to configure the target
74. Target type: AWS service
75. Select a target: SNS topic
76. Topic: Selected the SNS topic created in Step 8.2
77. Reviewed and created the rule

How the Alerting Pipeline Works End-to-End

1. A user performs a CreateBucket or DeleteBucket API call in the AWS account
2. CloudTrail captures the API call as a management event and logs it
3. EventBridge receives the CloudTrail event and evaluates it against all rules
4. The S3 bucket create/delete rule matches the event
5. EventBridge publishes a message to the SNS topic with the event details
6. SNS delivers the message as an email to all confirmed subscribers
7. The administrator receives an email alert within seconds of the S3 API call

This pipeline demonstrates a real-world cloud security monitoring pattern used in production AWS environments for change management alerts, compliance notifications, and security incident detection.

📷 **[SCREENSHOT]**

Amazon EventBridge Rule Created with S3 Event Pattern

Capture the EventBridge > Rules page showing the created rule, and the rule detail page showing: Event pattern (source: aws.s3, eventName: CreateBucket/DeleteBucket) and the Target (SNS topic ARN).

📷 **[SCREENSHOT]**

Email Alert S3 Bucket Event Notification Received

Capture the email inbox showing the SNS notification email received after creating or deleting a test S3 bucket. Show the email subject (containing 'CreateBucket' or 'DeleteBucket') and the JSON body showing the event details (account ID, bucket name, timestamp, principal).

9. Phase 7 KMS Encryption & DynamoDB

9.1 KMS Symmetric Key Creation

AWS KMS	Service: KMS > Customer managed keys > Create key > Symmetric, Encrypt and decrypt

A customer managed key (CMK) was created using AWS Key Management Service (KMS). Unlike AWS managed keys (which AWS controls), customer managed keys give the account owner full control over key policies, rotation, and deletion. This key was created specifically to encrypt the DynamoDB table created in Phase 9.2, providing encryption at rest with customer-managed key material.

78. Navigated to KMS > Customer managed keys > Create key
79. Key type: Symmetric (single key used for both encrypt and decrypt operations)
80. Key usage: Encrypt and decrypt
81. Key material origin: KMS (AWS generates and manages the key material)
82. Regionality: Single-Region key (sufficient for this lab)
83. Alias: [insert key alias e.g. alias/dynamodb-encryption-key]
84. Description: KMS key for DynamoDB table encryption
85. Key administrators: admin-user (the IAM user who can manage the key)
86. Key users: it-user-1, it-user-2 (IT group users who can use the key for encrypt/decrypt)
87. Reviewed the key policy and created the key noted the Key ARN

KMS Key Attribute	Value
Key Type	Symmetric
Key Usage	Encrypt and decrypt
Key Alias	alias/dynamodb-encryption-key
Key Administrators	admin-user (can manage/delete the key)
Key Users	it-user-1, it-user-2 (can use the key to encrypt/decrypt data)
Key Rotation	Automatic annual rotation enabled (best practice)
Key ARN	arn:aws:kms:[region]:[account-id]:key/[key-id]

📷 **[SCREENSHOT]**

AWS KMS Customer Managed Key Created

Capture the KMS > Customer managed keys page showing the created key with its alias (alias/dynamodb-encryption-key), key ID, key type (Symmetric), and status (Enabled). Also capture the key detail page showing the Key policy tab with admin-user as administrator and IT users as key users.

9.2 DynamoDB Table with KMS Encryption

Amazon DynamoDB	Service: DynamoDB > Tables > Create table > Encryption at rest: AWS managed key → Customer managed key

A DynamoDB table was created and linked to the KMS customer managed key created in Phase 9.1. This enables encryption at rest all data written to the table is encrypted using the CMK, and the CMK's key policy controls who can read that data. Without access to the KMS key, even AWS support cannot read the encrypted table contents.

88. Navigated to DynamoDB > Tables > Create table
89. Table name: [insert table name e.g. organisation-data]
90. Partition key: userId (String) or insert actual partition key used
91. Sort key: timestamp (String) optional, insert if used
92. Table settings: Customise settings
93. Read/Write capacity mode: On-demand (simplest for lab)
94. Encryption at rest: Scrolled to Encryption section
95. Selected: Customer managed key (CMK)
96. KMS key: Selected the key created in Phase 9.1 (alias/dynamodb-encryption-key)
97. Created the table DynamoDB created the table with KMS encryption enabled

📷 **[SCREENSHOT]**

AWS DynamoDB Table Created with KMS Customer Managed Key Encryption

Capture the DynamoDB > Tables page showing the created table, and the table detail page with the 'Additional settings' tab showing Encryption type = 'AWS KMS customer managed key' and the Key ARN of the key created in Phase 9.1.

9.3 User Access Control on DynamoDB

Inline policies were added to the relevant groups to define which users can access the DynamoDB table and what operations they can perform. IT users were granted full DynamoDB access, while content group users were given read-only access appropriate to their role.

IT Group Full DynamoDB Access

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "dynamodb:*",
      "Resource":
"arn:aws:dynamodb:[region]:[account-id]:table/organisation-data"
    },
    {
      "Effect": "Allow",
```

```
      "Action": ["kms:Decrypt", "kms:GenerateDataKey"],
      "Resource": "arn:aws:kms:[region]:[account-id]:key/[key-id]"
      // IT users need KMS decrypt permission to read encrypted table data
    }
  ]
}
```

Content Groups DynamoDB Read-Only Access

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["dynamodb:GetItem", "dynamodb:Query", "dynamodb:Scan"],
      "Resource":
"arn:aws:dynamodb:[region]:[account-id]:table/organisation-data"
    },
    {
      "Effect": "Allow",
      "Action": "kms:Decrypt",
      "Resource": "arn:aws:kms:[region]:[account-id]:key/[key-id]"
    }
  ]
}
```

📷 **[SCREENSHOT]**

AWS DynamoDB + KMS User Access Policy Applied to IT Group

Capture the IAM > User groups > IT > Permissions tab showing the DynamoDB inline policy with dynamodb: and kms:Decrypt/GenerateDataKey permissions. Also show the DynamoDB table overview confirming the KMS key is linked.*

10. IAM Least Privilege Summary

The following table provides a consolidated view of every access permission granted across all groups and services in this project, demonstrating the application of the principle of least privilege throughout.

Group	S3 Access	EC2 Access	DynamoDB	KMS	Notes
Facebook	Own bucket only (rw)	Own EC2 only	Read only	Decrypt only	Cannot access Instagram, Threads, or IT resources
Instagram	Own bucket only (rw)	Own EC2 only	Read only	Decrypt only	Cannot access Facebook, Threads, or IT resources
Threads	Own bucket only (rw)	Own EC2 only	Read only	Decrypt only	Cannot access Facebook, Instagram, or IT resources
IT	**ALL 3 buckets (full s3:*)**	**ALL 3 instances (ec2:*)**	**Full (dynamo:*)**	**Encrypt + Decrypt**	IT administrators cross-departmental access for management and support

11. Key Findings & AWS Security Best Practices

11.1 Key Findings

#	Finding	Detail
F-01	**Root account hardening is the foundation of AWS security**	Enabling MFA on root and creating an IAM admin user immediately reduces the most catastrophic risk in an AWS account unrestricted root credential compromise
F-02	**Inline policies enforce tight least-privilege at the group level**	Using inline policies (rather than managed policies) creates an explicit, auditable coupling between each group and its specific resource ARNs making access scope immediately visible in the group's Permissions tab
F-03	**IT group cross-account access is a common enterprise pattern**	Granting the IT group full access to all resources mirrors the real-world role of a cloud operations team they need visibility across all departmental resources for monitoring, incident response, and cost management
F-04	**EventBridge + CloudTrail + SNS is a powerful serverless monitoring stack**	The alerting pipeline (CloudTrail → EventBridge → SNS → Email) demonstrates a fully serverless, event-driven monitoring solution with no compute to manage a pattern used widely in production AWS environments for change management and security alerting
F-05	**Customer managed KMS keys provide cryptographic control over DynamoDB data**	Linking a CMK to DynamoDB means that revoking the KMS key policy (removing key users) instantly prevents decryption of table data even for users with DynamoDB full access adding a cryptographic access control layer beyond IAM
F-06	**Bedrock RAG architecture enables governed AI within the AWS boundary**	The knowledge base chatbot demonstrates that AI capabilities can be delivered to employees while maintaining full data sovereignty, compliance logging, and access control directly satisfying an AI Use Policy requirement

11.2 AWS Security Best Practices Demonstrated

- Enable MFA on root account immediately after account creation never use root for day-to-day operations
- Create IAM users and groups never share credentials or operate as root for routine tasks
- Apply the principle of least privilege grant only the minimum S3, EC2, DynamoDB, and KMS permissions required for each group's function
- Use inline policies for tightly scoped, group-specific resource access resource ARNs in policy statements provide the most granular access control
- Enable CloudTrail for all management events every API call in the account is auditable, supporting compliance, incident investigation, and change management
- Use customer managed KMS keys for sensitive data CMKs give the account owner cryptographic control that IAM alone cannot provide
- Deploy AI services within the AWS boundary Bedrock Knowledge Base keeps all data and queries within the organisation's account rather than external AI platforms

12. Screenshot Evidence Index

The following table provides a complete index of all screenshot placeholders in this document. Screenshots should be captured from the AWS Management Console during the live project build and inserted at the corresponding locations throughout the document.

#	Screenshot Caption	Section	AWS Service
1	Root Account MFA Assigned (Security Credentials Page)	3 Root MFA	AWS IAM
2	Admin User Created with AdministratorAccess Policy	4 Admin IAM User	AWS IAM
3	All Four Groups Created (User Groups List)	5.1 Groups	AWS IAM
4	All 8 Users Listed with Group Memberships	5.2 Users	AWS IAM
5	Three S3 Buckets Created (S3 Buckets List)	6.1 S3 Buckets	Amazon S3
6	Facebook Group Inline Policy (S3)	6.2 S3 Inline Policies	AWS IAM
7	Instagram & Threads Group Inline Policies (S3)	6.2 S3 Inline Policies	AWS IAM
8	IT Group Inline Policy: Full S3 Access	6.3 IT S3 Policy	AWS IAM
9	Three EC2 Instances Running (Instances List)	7.1 EC2 Instances	Amazon EC2
10	Group Inline Policies: EC2 Access (Facebook/Instagram/Threads/IT)	7.2 EC2 Policies	AWS IAM + EC2
11	CloudTrail Trail Created (Management Events)	8.1 CloudTrail	AWS CloudTrail
12	SNS Topic Created & Email Subscription Confirmed	8.2 SNS	Amazon SNS
13	EventBridge Rule: S3 Event Pattern & SNS Target	8.3 EventBridge	Amazon EventBridge
14	Email Alert Received S3 Bucket Event Notification	8.3 Alert Test	Email / SNS
15	KMS Customer Managed Key Created	9.1 KMS Key	AWS KMS
16	DynamoDB Table with KMS CMK Encryption	9.2 DynamoDB	Amazon DynamoDB
17	DynamoDB + KMS Access Policy Applied to IT Group	9.3 DynamoDB Access	AWS IAM + DynamoDB

End of Document